APLICACIONES INTERACTIVAS

# Gestor de Gimnasio

## Etapa 1 — Back-end

Documentación técnica

Java · Spring Boot · Spring Data JPA · MySQL · HTTP / JSON · Postman

Integrantes: Ignacio Ugarte, Mateo Galli

Docentes: Juan Ignacio López, Luciano Pueyo

Fecha de entrega: 24/09/2026

# 1. Introducción

El **Gestor de Gimnasio** es el back-end de un sistema para administrar **planes**, **clases**, **socios**, **reservas** y **pagos** de un gimnasio. Es un servidor que recibe peticiones HTTP, aplica la lógica de negocio y guarda los datos en una base MySQL. El cliente (Postman, y en la próxima etapa un front-end) nunca accede a la base: solo usa los endpoints de la API y el formato JSON.

## 1.1 Tecnologías

| Tecnología | Uso |
|---|---|
| Java 25 + Spring Boot 4 | Lenguaje y framework del servidor (Tomcat en el puerto 8080). |
| Spring Data JPA + Hibernate | Guarda y lee los objetos Java en la base sin escribir SQL. |
| MySQL (XAMPP) | Base de datos gimnasio_db. |
| Jakarta Validation | Valida los datos de entrada. |
| Lombok | Genera getters y setters. |
| Postman | Prueba y demostración de la API. |

## 1.2 Proceso de desarrollo

1. Se definieron las 5 entidades y sus relaciones, tomando como base el ejemplo de clase (Alumno – Materia – Carrera).
2. Se programaron las entidades, luego los repositories, los services y por último los controllers.
3. Se agregaron validaciones y un manejador global de errores.
4. Se conectó MySQL (XAMPP) y se escribió el script SQL con datos de ejemplo.
5. Se probó todo con una colección de Postman de 35 peticiones, todas con resultado correcto.

# 2. Arquitectura en capas

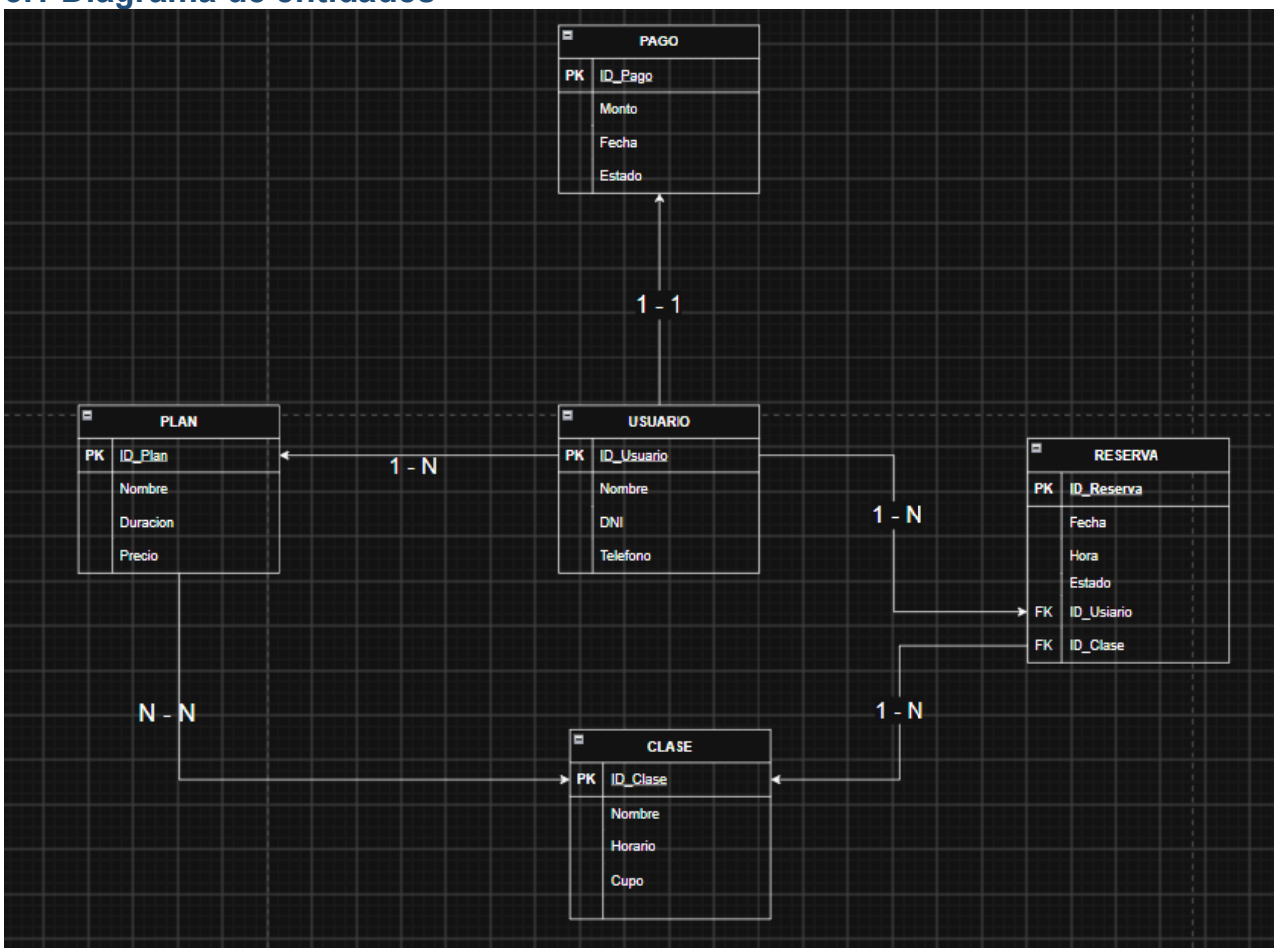
```
Cliente (Postman) —HTTP/JSON→ CONTROLLER → SERVICE → REPOSITORY —SQL→ MySQL
```

| Capa | Responsabilidad |
|---|---|
| controller | Recibe la petición HTTP, valida los datos (@Valid) y devuelve la respuesta con su código de estado. |
| service | Lógica de negocio: verifica que existan los datos relacionados y aplica las reglas del gimnasio. |
| repository | Acceso a la base. Spring genera las consultas SQL automáticamente. |
| model | Entidades: cada clase es una tabla y cada atributo una columna. |
| exception | Convierte los errores en respuestas HTTP claras (400, 404, 409). |

Cada capa solo usa a la de abajo (Controller → Service → Repository), así cada una tiene una única responsabilidad y el mismo back-end puede usarlo cualquier front-end.

# 3. Modelo de datos

## 3.1 Diagrama de entidades




## 3.2 Relaciones

| Relación | Tipo | Implementación |
|---|---|---|
| Usuario → Plan | N a 1 | Muchos Usuarios pueden tener el mismo plan. |
| Reserva → Usuario | N a 1 | Un usuario puede hacer muchas reservas. |
| Reserva → Clase | N a 1 | Una clase puede recibir muchas reservas. |
| Pago → Usuario | N a 1 | Un usuario puede hacer muchos pagos. |
| Plan ↔ Clase | N a N | Un plan incluye varias clases y una clase puede estar en varios planes. |

Las relaciones son unidireccionales: solo la entidad que tiene la clave foránea conoce a la otra (Usuario conoce su Plan, pero Plan no guarda la lista de usuarios). Esto simplifica el código y evita que el JSON entre en un ciclo infinito (plan → usuarios → plan → …).

Reserva funciona como el nexo entre Usuario y Clase: además de unirlos, guarda un dato propio (la fecha a la que asiste el usuario). Por eso es una entidad con dos @ManyToOne y no un @ManyToMany.

## 3.3 Base de datos

El script `database/gimnasio_db.sql` crea la base, las 6 tablas (con claves primarias, foráneas y restricciones UNIQUE) y carga datos de ejemplo. Además, con `ddl-auto=update`, Hibernate crea las tablas automáticamente a partir de las entidades si no existen.

## 4. Responsabilidades de las clases

| Clase | Responsabilidad |
|---|---|
| GestorGimnasioApplication | Método main: arranca Spring y el servidor. |
| Plan, Clase, Socio, Reserva, Pago | Entidades con sus columnas, relaciones y validaciones. |
| *Repository (5) | Heredan de JpaRepository: findAll, findById, save, deleteById, existsById. SocioRepository agrega findByActivo y ReservaRepository agrega countByClaseIdAndFecha. |
| PlanService | CRUD de planes; agregar y quitar clases de un plan (N a N). |
| ClaseService | CRUD de clases. |
| SocioService | CRUD de socios; alta/baja lógica; completa la fecha de ingreso y busca el plan. |
| ReservaService | CRUD de reservas; controla que el socio esté activo y que haya cupo. |
| PagoService | CRUD de pagos; completa la fecha si no se envía. |
| *Controller (5) | Exponen los endpoints REST de cada entidad. |
| GlobalExceptionHandler | Atrapa los errores y responde 400, 404 o 409. |

## 5. Sintaxis principal

| Anotación | Qué hace |
|---|---|
| @Entity / @Table | La clase es una tabla de la base / nombre de la tabla. |
| @Id / @GeneratedValue | Clave primaria, generada automáticamente. |
| @Column | Configura la columna (obligatoria, única, largo). |
| @ManyToOne + @JoinColumn | Relación N a 1 y nombre de la clave foránea. |
| @ManyToMany + @JoinTable | Relación N a N y su tabla intermedia. |
| @RestController / @RequestMapping | Clase que atiende peticiones HTTP / ruta base. |
| @GetMapping, @PostMapping, @PutMapping, @PatchMapping, @DeleteMapping | Asocian un método a un verbo HTTP. |
| @PathVariable / @RequestParam / @RequestBody | Toman datos de la ruta / de la URL / del JSON. |
| @Valid + @NotBlank, @NotNull, @Email, @Positive | Validación de los datos de entrada. |
| @Service / @Transactional | Clase de negocio / operación atómica (si falla, no se guarda nada). |
| @RestControllerAdvice / @ExceptionHandler | Manejo centralizado de errores. |
| @Data (Lombok) | Genera getters, setters, toString, equals y hashCode. |

Ejemplo de relación N a N en `Plan`:

```
@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(name = "plan_clase",
    joinColumns = @JoinColumn(name = "plan_id"),
    inverseJoinColumns = @JoinColumn(name = "clase_id"))
```

```
private List<Clase> clases = new ArrayList<>();
```

Ejemplo de endpoint: `Optional` permite responder 404 si el id no existe, y `ResponseEntity` define el código HTTP.

```
@GetMapping("/{id}")
public ResponseEntity<Socio> getById(@PathVariable Long id) {
    Optional<Socio> encontrado = socioService.findById(id);
    if (encontrado.isPresent()) {
        return ResponseEntity.ok(encontrado.get());   // 200
    }
    return ResponseEntity.notFound().build();         // 404
}
```

# 6. Endpoints

Base: `http://localhost:8080`. `{entidad}` = `planes`, `clases`, `socios`, `reservas` o `pagos`.

| Método | Ruta | Qué hace | Respuestas |
|---|---|---|---|
| GET | /api/{entidad} | Lista todos | 200 |
| GET | /api/{entidad}/{id} | Busca uno | 200, 404 |
| POST | /api/{entidad} | Crea | 201, 400, 404, 409 |
| PUT | /api/{entidad}/{id} | Modifica todos los datos | 200, 400, 404 |
| DELETE | /api/{entidad}/{id} | Borra | 204, 404, 409 |
| PATCH | /api/socios/{id}/estado?activo=false | Alta/baja lógica del socio | 200, 404 |
| GET | /api/socios/filtro?activo=true | Filtra socios por estado | 200 |
| POST | /api/planes/{planId}/clases/{claseId} | Agrega una clase al plan | 200, 404 |
| DELETE | /api/planes/{planId}/clases/{claseId} | Quita una clase del plan | 200, 404 |

Las relaciones se envían con el id del objeto relacionado:

```
POST /api/socios
{ "dni": "40123456", "nombre": "Juan", "apellido": "Pérez",
  "email": "juan@mail.com", "plan": { "id": 1 } }

POST /api/reservas
{ "socio": { "id": 1 }, "clase": { "id": 1 }, "fecha": "2026-10-05" }
```

# 7. Lógica de negocio y errores

| Regla | Si no se cumple |
|---|---|
| Todo socio nuevo empieza activo; si no se envía fecha de ingreso, se usa la de hoy. | — |
| Los datos relacionados (plan, socio, clase) tienen que existir. | 404 |
| Un socio inactivo no puede reservar. | 409 |
| Una clase no acepta más reservas que su cupo para una misma fecha. | 409 |
| DNI y email no se repiten; no se borra un plan con socios ni una clase con reservas. | 409 |
| Datos obligatorios o con formato inválido. | 400 |

Los errores devuelven un JSON claro, por ejemplo: `{ "error": "El socio está inactivo y no puede reservar" }`.

# 8. Ejecución y pruebas

1. Iniciar MySQL desde XAMPP (usuario root, sin contraseña).
2. (Opcional) Importar `database/gimnasio_db.sql` para cargar datos de ejemplo.
3. Ejecutar `GestorGimnasioApplication` y esperar "Started GestorGimnasioApplication".
4. Importar en Postman `postman/gestor_gimnasio.postman_collection.json` y ejecutarla con "Run collection".

La colección tiene 35 peticiones en 8 carpetas: el CRUD de cada entidad, la relación N a N, el PATCH, los casos de error (400, 404 y 409) y el borrado final. Los ids creados se guardan solos en variables.

# 9. Decisiones de diseño

| Decisión | Justificación |
|---|---|
| Arquitectura en capas | Separa comunicación, negocio y datos: código más claro y reutilizable por cualquier front-end. |
| Modelo basado en el ejemplo de clase | Plan–Clase replica Alumno–Materias (N a N) y Socio–Plan replica Materia–Carrera (N a 1). |
| Entidades sin DTOs, relaciones unidireccionales | Menos clases y un sistema más simple, sin ciclos en el JSON. |
| PATCH para el estado del socio | Es una modificación parcial; permite la baja lógica sin perder el historial. |
| Optional + ResponseEntity | Permiten responder 404 en lugar de devolver null o un error 500. |
| Manejo de errores centralizado | Una sola clase traduce los errores a códigos HTTP, sin try/catch en los controllers. |
| @Transactional | Si una operación falla a mitad de camino, no se guarda nada a medias. |